

X

Genel

Duyurular

LAB

Lab 2

5 December Lab

11 December Lab

25 December Final Exam

CSE101L / Genel / LAB



- Description
- Submission view

Available from: Perşembe, 21 Kasım 2024, 9:15 AM
Due date: Perşembe, 21 Kasım 2024, 12:35 PM
Maximum number of files: 3
Type of work: Individual work

Question 1:

Given a positive integer $n > 1$, write an algorithm to find its prime factor decomposition. The result should be returned as a string in the following format:

Output Format:

- The result should be a string:
 - "(p1**n1)(p2**n2)...(pk**nk)"
 - p1,p2,...,pk: Prime factors of the number n, sorted in ascending order.
 - n1,n2,...,nk: Exponents of each prime factor.
 - If the exponent $n(i)=1$, omit the ****1** part and only include the prime factor.

Examples:

- Input:** $n=86240$
Output: "(2**5)(5)(7**2)(11)"
- Input:** $n=360$
Output: "(2**3)(3**2)(5)"

Implementation Details

- Class Name:** PrimeFactorDecomposition
- Method Name:** decompose
- Method Signature:**
 - Input:**
 - int n: A positive integer ($n > 1$).
 - Output:**
 - String: The prime factor decomposition in the required format.

Question 2:

Write a function that takes a non-negative integer as input, representing a total number of seconds, and returns a string that formats this duration into a human-readable time format.

Output Format:

The function should return a string in the following format:

- HH:MM:SS
 - HH: Hours, always 2 digits, ranging from 00 to 99.
 - MM: Minutes, always 2 digits, ranging from 00 to 59.
 - SS: Seconds, always 2 digits, ranging from 00 to 59.

Note:

- Each component must be padded with zeros to ensure it is always 2 digits.

Requirements and Rules:

- Input:**
 - The function will take a non-negative integer ($0 \leq \text{seconds} \leq 359999$).
 - This integer represents the total number of seconds.
- Constraints:**
 - The maximum hours value is $HH=99$.
 - The minute and second values MM and SS cannot exceed 59.
- Function Behavior:**
 - If the input is 0, the output should be 00:00:00.
 - If the input is greater than 0, the total seconds should be broken down into hours, minutes, and seconds, and formatted accordingly.

Requirements and Rules:

- Class Name:** HumanReadableTime
- Method Name:** makeReadable
- Parameters:**
 - Input:**
 - int seconds: The total number of seconds (a non-negative integer between 0 and 359999).
 - Output:**
 - String: A string in the format HH:MM:SS.

Question 3:

You need to implement the **Rail Fence Cipher** in two functions: one for encoding and the other for decoding. This cipher works by placing characters diagonally along a series of "rails" (rows) in a zigzag pattern. The string is encoded by reading each rail (row) from left to right.

Rules and Details:

- Encoding:**
 - Start by moving diagonally downward, placing one character per rail.
 - When the bottom rail is reached, reverse direction and move diagonally upward.
 - Continue this zigzag pattern until the end of the string.
 - After completing the zigzag, concatenate the characters on each rail from top to bottom to form the encoded string.
- Decoding:**
 - Start with the encoded string and determine how many characters belong to each rail based on the zigzag pattern.
 - Populate each rail with the corresponding characters from the encoded string.
 - Traverse the zigzag pattern to reconstruct the original string.
- Examples:**
 - Input: "WEAREDISCOVERDFLEEATONCE", number of rails = 3
 - Rail pattern:

W	E		C		R		L		T		E
	E	R	D	S	O		E	E	F		E
		A		I		V		D		E	N

 - Encoded string: "WECRLTEERDSOEFEAOCAVDEN"
 - Decoded string: "WEAREDISCOVERDFLEEATONCE"
- Constraints:**
 - Number of rails (≥ 2).
 - Empty strings should return an empty string.
 - Do not filter out punctuation or spaces — they are part of the input and output.

Requirements and Rules:

- Class Name:** RailFenceCipher
- Method Names:**
 - encode: Encodes the input string using the Rail Fence Cipher.
 - decode: Decodes the encoded string back to its original form.
- Method Signatures:**
 - public static String encode(String text, int rails)
 - Input:**
 - text: The string to encode (e.g., "WEAREDISCOVERDFLEEATONCE").
 - rails: The number of rails (e.g., 3).
 - Output:**
 - The encoded string (e.g., "WECRLTEERDSOEFEAOCAVDEN").
 - public static String decode(String encodedText, int rails)
 - Input:**
 - encodedText: The encoded string (e.g., "WECRLTEERDSOEFEAOCAVDEN").
 - rails: The number of rails (e.g., 3).
 - Output:**
 - The decoded string (e.g., "WEAREDISCOVERDFLEEATONCE")